HANDS-ON WORKSHOP

Advanced Python

OOP, data, databases, concurrency and APIs, built as one running project

About 70% hands-on · 1 Orders Service

DAY 1

Core Python and the data pipeline

#	Module
0	Setup
1	Object-oriented programming
2	Language features and best practices
3	Exception handling
4	Requests
5	Pandas
6	Pydantic
7	Integration: the ingest pipeline

DAY 2

Persistence, concurrency and APIs

#	Module
8	Day 1 recap and Postgres up
9	Psycopg
10	SQLAlchemy ORM
11	Async and await
12	Parallel programming
14	FastAPI
15	Capstone build and demos

THE RUNNING PROJECT

One service, built over two days

DAY 1 · INGEST

MOD 1-3
Domain
Dataclasses, utilities, errors

→

MOD 4
Fetch
Requests client

→

MOD 5
Clean
Pandas analytics

→

MOD 6-7
Validate
Pydantic and ingest.py

DAY 2 · PERSIST AND SERVE

MOD 9
Raw SQL
Psycopg and Postgres

→

MOD 10
ORM
SQLAlchemy repository

→

MOD 11-12
Speed
Async and parallel

→

MOD 14
API
FastAPI endpoints

DAY 1

Core Python and the data pipeline

Fetch, clean and validate order data with a command you wrote

Get the environment running

Environment

- Python 3.11+ and a venv
- make setup installs everything
- VS Code or PyCharm

Services

- Postgres in Docker (from Day 2)
- Local mock API on port 8001
- Flaky, slow, paginated endpoints

Starter repo

- app/ and tests/ layout
- Dirty orders CSV
- make test-lab LAB=NN

Java equivalents

venv + pip → Maven / Gradle: Per-project dependencies; pyproject.toml is your pom.xml.

python → java + JVM: Runs the code directly, with no separate compile step.

make setup / make test → mvn install / mvn test: Same idea.

app/ and tests/ → src/main and src/test: The same code and test split.

Object-oriented programming

Classes

- Instance, class, static methods
- @property with setters
- __repr__, __eq__, __lt__
- __hash__, __len__, __iter__

Inheritance

- super() and the MRO
- Composition over inheritance
- ABCs and Protocols
- SOLID in Python terms

Dataclasses and patterns

- field, frozen, slots
- __post_init__
- Repository pattern
- Strategy pattern

Java equivalents

__eq__ / __hash__ → equals() / hashCode(): The same contract applies.

__repr__ → toString(): Developer-readable text for an object.

Protocol → interface: Structural: any class with the methods fits, no implements.

@property → getter / setter: Field-style access (order.total) backed by a method.

LAB 1

Build the domain model

1. Model Customer, Product, Order and OrderLine as frozen dataclasses
2. Add an abstract DiscountPolicy
3. Implement percentage and bulk discounts with Decimal
4. Expose Order.total as a property
5. Make orders sortable and add a Protocol-based grand_total

You finish with

A domain model reused in every later lab.

Java equivalents

frozen dataclass → record: Immutable, with equals, hashCode and toString generated.

ABC + @abstractmethod → abstract class: Same role.

Decimal → BigDecimal: Exact money arithmetic; never float or double.

__lt__ → Comparable.compareTo: Lets sorted() order your objects.

Language features and best practices

Language

- Type hints, generics, TypedDict
- Decorators and closures
- Generators and itertools
- Context managers

Modern syntax

- match statement
- enum and pathlib
- f-strings, walrus operator
- mypy or pyright run

Practice

- logging, not print
- Config from env vars
- pytest fixtures
- ruff, black, pre-commit

Java equivalents

Type hints → static types: Optional, checked by mypy, not enforced at runtime.

Decorator → annotation + AOP aspect: A function that wraps another function.

Generator → Iterator / Stream: Produces values lazily, one at a time.

with → try-with-resources: __enter__ / __exit__ play the AutoCloseable role.

LAB 2

Reusable utilities

1. Write @timed and @retry(times, delay) decorators
2. Write a context manager that logs duration
3. Write a generator that streams a large orders CSV
4. Write a batching helper
5. Add pytest tests for the domain classes

You finish with

A utilities module used by the error handling and Requests labs.

Java equivalents

@retry → Spring @Retryable: Same idea, written by hand as a wrapper function.

Context manager → try-with-resources: Cleanup is guaranteed, even on errors.

CSV generator → BufferedReader.lines(): Streams a big file without loading it all.

pytest → JUnit 5: Plain assert statements; fixtures replace @BeforeEach.

Exception handling

Flow

- try, except, else, finally
- Catch specific exceptions
- raise ... from chaining
- Bare raise to re-raise

Design

- AppError hierarchy
- NotFoundError, ValidationError
- ExceptionGroup and except*
- logger.exception

Anti-patterns

- Bare except
- Swallowed errors
- Exceptions as control flow
- Fail fast vs degrade gracefully

Java equivalents

try / except / finally → try / catch / finally: else runs only if no error.

raise X from Y → new X(msg, cause): Keeps the original exception as the cause.

No checked exceptions: Every Python exception behaves like a RuntimeException; no throws clause.

AppError hierarchy → custom exception classes: Same pattern.

LAB 3

Robust error handling

1. Build the AppError hierarchy
2. Add OutOfStockError and InvalidQuantityError
3. Write an import routine that collects per-row failures
4. Apply @retry to a flaky function
5. Log failures with logger.exception

You finish with

An import report listing every bad row, not just the first.

Java equivalents

OutOfStockError → custom RuntimeException: Named after the business rule.

logger.exception → log.error(msg, e): Logs the message with the full stack trace.

Per-row failures → catch inside the loop: Collect errors in a list and continue.

@retry → Resilience4j Retry: Re-runs a flaky call after a delay.

HTTP with Requests

HTTP essentials

- Methods, status codes, headers
- params and json=
- Always set a timeout
- raise_for_status()

Robust clients

- Session reuse and auth
- Retry with backoff
- 429 and Retry-After
- Pagination patterns

Test and compare

- requests-mock or responses
- Map HTTP errors to AppError
- Streaming large downloads
- httpx comparison

Java equivalents

requests → HttpClient / RestTemplate: The standard blocking HTTP client.

Session → one shared HttpClient: Reuses connections, headers and auth.

raise_for_status() → error handler: Turns 4xx/5xx responses into exceptions.

httpx → WebClient: One client for both sync and async calls.

LAB 4

Build a product feed client

1. Wrap a Session with a timeout, retries and backoff
2. Page through the whole product feed lazily
3. Parse each item into a Pydantic-ready dict
4. Raise domain exceptions on failure
5. Write three tests with requests-mock

Local mock API

Flaky, slow, paginated and rate-limited endpoints let you trigger failures on demand, no internet needed.

Java equivalents

requests-mock → WireMock / MockRestServiceServer: Fake HTTP in tests.

Lazy page generator → custom Iterator: Fetches the next page only on demand.

Retry with backoff → Resilience4j Retry: Exponential waits between attempts.

Parsed item dict → DTO: Plain data handed on to validation.

Pandas

Core

- Series and DataFrame
- read_csv, read_json, read_sql
- loc, iloc, boolean masks
- query

Clean and transform

- isna, fillna, dropna
- astype, drop_duplicates
- assign, groupby and agg
- merge, pivot_table

Performance and pitfalls

- Vectorize before apply
- Categorical dtype
- Chunked reading
- SettingWithCopyWarning

Java equivalents

DataFrame → in-memory SQL table: No JDK type; closest are Tablesaw or a Spark Dataset.

groupby + agg → Collectors.groupingBy: Group and aggregate, per column.

Vectorized ops → Stream map over a column: Runs in native code; avoid row loops.

merge → SQL JOIN: Combines two tables on a shared key.

LAB 5

Sales analytics

1. Clean nulls, duplicates, prices and four date formats
2. Reject bad rows with a reason column
3. Compute revenue by month
4. Find the top 10 customers
5. Merge with the products table
6. Export a summary report to CSV

Starter data

The orders CSV is deliberately dirty, with known defect counts the tests check.

Java equivalents

dropna / fillna → null checks / Optional.orElse: Explicit missing-value rules.

drop_duplicates → Stream.distinct(): Removes repeated rows by key.

to_datetime(format=) → DateTimeFormatter: Parse with an explicit pattern.

to_csv → Apache Commons CSV / OpenCSV: Writes the summary report.

Pydantic

Models

- BaseModel and Field constraints
- Coercion vs strict mode
- Nested models and defaults
- default_factory

Validation

- field_validator
- model_validator
- ValidationError handling
- Pydantic vs dataclasses

I/O and config

- model_dump, model_dump_json
- model_validate
- Aliases
- pydantic-settings

Java equivalents

BaseModel + Field → Bean Validation DTO: Like @NotNull, @Min and @Size.

model_dump_json / model_validate_json → Jackson: Same job.

Field(alias=) → @JsonProperty: The external JSON name differs from the field name.

pydantic-settings → @ConfigurationProperties: Typed config from env vars.

LAB 6

Validated schemas

1. Define OrderCreate, OrderRead and ProductIn
2. Add constraints and a cross-field check
3. Validate a mixed set of good and bad JSON payloads
4. Produce a readable error report
5. Add a Settings class for configuration

You finish with

Schemas reused directly by the ingest pipeline and the FastAPI endpoints.

Java equivalents

OrderCreate / OrderRead → request / response DTOs: Separate in and out.

model_validator → class-level ConstraintValidator: For cross-field rules.

ValidationError.errors() → Set<ConstraintViolation>: Every problem at once.

Settings class → @ConfigurationProperties: Loaded from env vars or .env.

LAB 7

Day 1 integration: the ingest pipeline

1. Fetch products with the Requests client
2. Load and clean the orders CSV
3. Validate order payloads with Pydantic
4. Write clean, rejects, summary and report files
5. Run it as a command-line tool

You finish with

```
python scripts/ingest.py
--orders data/orders_dirty.csv
```

Exit code 0 on success, 2 when the service is unreachable.

Java equivalents

scripts/ingest.py → main class in a JAR: The entry point run from a terminal.

argparse → picocli: Parses command-line options.

sys.exit(2) → System.exit(2): The same exit-code contract.

logging → SLF4J + Logback: Leveled logs instead of print.

DAY 2

Persistence, concurrency and APIs

Store the data in Postgres, make it fast, and serve it over HTTP

LAB 8

Day 1 recap and Postgres up

1. Start Postgres with make db-up
2. Run scripts/check_db.py
3. Run make test-day DAY=1 to confirm Day 1 is green
4. Behind? Check out the day2-start tag

You finish with

A running database and a clean starting point for everyone.

Java equivalents

make db-up → docker compose / Testcontainers: The same Postgres container.

check_db.py → Actuator /health: Confirms the database is reachable.

make test-day → mvn test with a tag filter: Runs one day's tests.

day2-start tag → Git tag: Same Git; nothing Python-specific.

Psycopg 3

Basics

- Connections and cursors
- Context managers
- Row factories: dict_row, class_row

Safety

- Parameterized queries
- Live SQL injection demo
- Transactions and rollback
- conn.transaction()

Scale

- executemany
- COPY for bulk loads
- psycopg_pool
- Async connection preview

Java equivalents

psycopg → JDBC driver: Connection and cursor map to Connection and Statement / ResultSet.

%s placeholders → PreparedStatement ?: Parameters stop SQL injection.

conn.transaction() → @Transactional: Commits on success, rolls back on error.

psycopg_pool → HikariCP: Connection pooling.

LAB 9

Raw SQL data layer

1. Create the tables
2. Bulk load the cleaned CSV with COPY
3. Write get_order and top_customers
4. Write create_order as one transaction that rolls back if out of stock
5. Return Pydantic models via class_row

You finish with

A working data layer you replace with the ORM in the next module.

Java equivalents

COPY → PgJDBC CopyManager: The same Postgres bulk-load command.

with conn.transaction() → @Transactional: Rolls back on exception.

class_row → RowMapper: Maps each row to an object.

executemany → addBatch / executeBatch: Many rows in one round trip.

SQLAlchemy 2.0 ORM

Models

- DeclarativeBase, Mapped
- mapped_column
- One-to-many, many-to-many
- back_populates, cascades

Querying

- select, where, join
- Session and unit of work
- scalars()
- Transactions

Performance

- The N+1 problem
- selectinload, joinedload
- Alembic migrations (demo)
- ORM vs raw SQL

Java equivalents

SQLAlchemy ORM → JPA / Hibernate: Maps classes to tables.

Mapped / mapped_column → @Entity / @Column: Declares mapped fields.

Session → EntityManager: Unit of work; tracks changes until commit.

selectinload / joinedload → JOIN FETCH: Same N+1 fix as in Hibernate.

LAB 10

ORM and repository

1. Re-implement the data layer as ORM models
2. Add a Repository class
3. Reproduce an N+1 problem using a statement counter
4. Fix it with selectinload
5. Write an Alembic migration that adds a column

You finish with

The same behavior as Module 9, behind a clean repository interface.

Java equivalents

Repository class → Spring Data repository: Written by hand; no generated methods.

relationship(back_populates) → @OneToMany(mappedBy): Same pattern.

Statement counter → Hibernate Statistics: Counts SQL statements.

Alembic → Flyway / Liquibase: Versioned schema migrations.

Async and await

Foundations

- I/O-bound vs CPU-bound
- Event loop and coroutines
- async def and await
- create_task, gather

Tools

- TaskGroup (3.11)
- Timeouts and cancellation
- httpx.AsyncClient
- Async SQLAlchemy

Pitfalls

- Blocking calls in coroutines
- asyncio.to_thread
- Semaphores for rate limits
- Errors that cancel siblings

Java equivalents

async / await → CompletableFuture chains: Written like sequential code.

Event loop → Netty / WebFlux event loop: One thread, many I/O tasks.

asyncio tasks → virtual threads (Java 21): Cheap I/O concurrency, but Python must await.

asyncio.to_thread → supplyAsync(..., executor): Offloads blocking work.

LAB 11

Async fan-out

1. Fetch items sequentially, then with gather, and compare timings
2. Add a semaphore limit
3. Handle errors without cancelling the whole batch
4. Handle timeouts per request
5. Convert one repository method to async SQLAlchemy

You finish with

Measured timings for sequential vs concurrent fetching, reused in the next lab.

Java equivalents

gather → CompletableFuture.allOf: Waits for all and collects the results.

asyncio.Semaphore → j.u.c.Semaphore: Caps how many calls are in flight.

asyncio.timeout → orTimeout(): A time limit on each call.

return_exceptions=True → exceptionally(): Keeps failures as values.

Parallel programming

Foundations

- The GIL, explained
- Threads for I/O-bound work
- Processes for CPU-bound work
- Free-threaded Python (experimental)

Tools

- threading: Lock, Queue
- ThreadPoolExecutor
- ProcessPoolExecutor
- multiprocessing.Pool

Pitfalls

- Race conditions
- Deadlocks
- Pickling cost and chunking
- spawn vs fork

Java equivalents

GIL → no Java equivalent: Java threads run on all cores; CPython threads take turns.

ThreadPoolExecutor → ExecutorService: The same submit / Future model.

threading.Lock → synchronized / ReentrantLock: RLock is the reentrant one.

ProcessPoolExecutor → separate JVMs: How Python uses all cores.

LAB 12

The benchmark showdown

1. Fetch pages sequentially, with threads and with async; compare
2. Run a CPU-bound task sequentially, with threads and with processes
3. Fix a race condition with a Lock
4. Process the large orders CSV in chunks with a process pool
5. Write a short 'which tool when' conclusion

You finish with

Measured results and a decision guide: threads help I/O, processes help CPU, async scales I/O.

Java equivalents

Threads on CPU work → unlike Java: Java threads speed it up; Python's don't (GIL).

Process pool → ForkJoinPool's role: Python's way to use every core.

Benchmark → JMH: Measure before you choose.

Lock around shared state → synchronized block: Fixes the race.

FastAPI

Core

- Routers, path, query, body
- Status codes
- response_model with Pydantic
- Depends for DB sessions

Behavior

- Async vs sync endpoints
- HTTPException
- Custom exception handlers
- Middleware, background tasks

Ship it

- OpenAPI docs at /docs
- TestClient
- Running with uvicorn
- Deployment notes

Java equivalents

APIRouter + @router.get → @RestController + @GetMapping: Same role.

Depends → constructor injection: FastAPI's version of @Autowired, per request.

Exception handler → @ControllerAdvice: Maps domain errors to HTTP.

uvicorn → embedded Tomcat / Netty: The server that runs the app.

LAB 14

Orders API

1. Implement the four endpoints
2. Wire the SQLAlchemy session through Depends
3. Map domain errors to 404, 409 and 422
4. Add request ID middleware and a background task
5. Write TestClient tests for success and error paths

Endpoints

```
POST /orders
GET /orders/{id}
GET /orders?customer_id=
GET /reports/top-customers
```

Java equivalents

Path / query params → @PathVariable / @RequestParam: Read from the URL.

Pydantic body → @RequestBody @Valid: Bad input gets 422 (Spring sends 400).

Middleware → Servlet Filter: Runs around every request.

TestClient → MockMvc: In-process API tests.

LAB 15

Capstone build and demos

1. Run make demo end to end and fix what breaks
2. Complete the architecture write-up
3. Add one improvement: JWT auth, Redis cache or rate limiting
4. Write at least two tests for it
5. Run make check and make test, then demo your API

make demo runs

Postgres, migrations, mock API, ingest pipeline, database load, API server and a smoke test.

Java equivalents

JWT dependency → Spring Security filter: Verifies the bearer token.

Redis cache → @Cacheable backed by Redis: Evict the cache on write.

Rate limiting → Bucket4j: Returns 429 over the limit.

make check / make test → mvn verify: Lint, type checks and tests.

WRAP-UP

The layered architecture

API	FastAPI routers and exception handlers
Schemas	Pydantic request and response models
Services	Business rules and domain exceptions
Repositories	SQLAlchemy sessions and queries
Database	PostgreSQL and Alembic migrations

BEST-PRACTICES CHECKLIST

Take these back to work

- Type-hint public functions
- Catch specific exceptions only
- Set a timeout on every HTTP call
- Parameterize every query
- Validate at the boundary

- Eager-load to avoid N+1
- Never block the event loop
- Measure before parallelizing
- Log, do not print
- Test with pytest, lint with ruff

Demos and questions

Run your Orders API, then pick one improvement to make next.